

CS3160 Mid-Semester Lab Exam

Duration: 2.5 hours

Before you start

There are two independent parts in this exam.

Part	What	Marks
1	RISC-V assembly (implement a circular buffer)	40
2	Extend the single cycle simulator (add support for Control and status registers)	60

Fill in `details.txt` first — name, roll number and machine number. At the end of the exam, save the work in your folder, lock the computer and leave the lab.

Getting Started

```
source /opt/cs3160/setup.sh
make test                both parts
make unit                just part 2's unit tests
python3 tests/run_tests.py part1  just the assembly
python3 tests/run_tests.py part2  just the simulator
```

`make test` will be used to grade your submission. So ensure your code works against it.

1 Part 1: Implement a circular buffer using RISC-V assembly

A circular buffer is a fixed-size FIFO (queue), implemented using a fixed-size array. The elements are inserted into the buffer at one end (say `tail`) and are removed from the other end (say `head`). When the write position runs off the end of the array it wraps back to the start of the array, and so the storage is reused forever. Note that the elements themselves do not have to be shifted on each insert or delete operation (see Section 1).

Implement a fixed size circular buffer (default capacity/`CAP` is 8) in `programs/asms/1-fifo.s`. Specifically, you are required to implement three functions:

`enqueue(value)`

- This is the insert operation which adds `value` as the newest/latest element into the buffer, and returns 0. If the buffer is already full, **the oldest element is silently overwritten** (overflow case) and returns `-1` instead. The buffer stays full but the oldest element disappears. You only have to return `-1`, and do not have to print anything or terminate the program.

`dequeue()`

– This is the delete operation which removes and returns the oldest element. If a buffer is **empty**, return `-1` (underflow case). This is also not an error, and so you do not have to print anything, or terminate the program.

`get_vacancy()`

– This operation returns how many words of the buffer are free (**CAP** when it is empty, 0 when it is full). It should not modify the contents of the buffer. Note that an enqueue into a full buffer does not change the value returned by this function (a value was overwritten, but not freed).

The buffer is empty at the start (no elements in it). **You are required to implement these three functions.**

Implementation Guidelines

The file already contains a `main` that drives them, and you should not change it: for each `i` from 0 to `n_ops-1` it reads `op[i]`, calls the function it names, and records **what that function returned** in `out[i]`.

Global	Size	Meaning
<code>n_ops</code>	1 word	how many operations to perform
<code>op</code>	24 words	1 = enqueue, 2 = dequeue, 3 = get_vacancy
<code>arg</code>	24 words	the value to enqueue; ignored by the other two
<code>out</code>	24 words	what each operation returned
<code>buf</code>	8 words	the storage
<code>head</code>	1 word	the index into <code>buf</code> of the oldest element
<code>count</code>	1 word	how many elements are in the buffer, 0 to 8

The first four variables are used by the harness and the driver. The last three rows are the state of the buffer, `enqueue` and `dequeue` will read or write them. All these three are at 0 at the beginning (corresponds to an empty buffer).

The conventional way of using a head and tail pointer may not work here. The description of enqueue and dequeue functions above allows the head and tail to point to the same location within the buffer in two scenarios: (i) when the buffer is completely empty, and (ii) when the buffer is completely full. Therefore, the suggested way is to derive the next free slot by calculating $(\text{head} + \text{count}) \bmod \text{CAP}$. This simplifies the implementation of `get_vacancy()`.

Follow the calling convention from week 2. `enqueue` is handed its value in `a0`; all three functions return their answer in `a0`. The driver keeps nothing in a register across a call, so all three functions are free to use any register they like except `sp` — and if you call a function of your own from inside one of them, save `ra` first.

► *Do not rename, reorder or resize the globals. The tests overwrite `n_ops`, `op` and `arg` with their own data and then read `out` back, so each of those arrays must keep room for all 24 words. Entries of `out` beyond `n_ops` must be left alone — which the driver already sees to.*

A worked example

With `n_ops = 8` and the script

enq 10, enq 20, enq 30, deq, deq, enq 40, deq, deq

out must end up holding

[0, 0, 0, 10, 20, 0, 30, 40, 0, 0, ...]

The three enqueues record 0. The first two dequeues give 10 and 20, the oldest first. The buffer then holds only 30; **enq 40** records 0, and the last two dequeues give 30 and then 40.

The cases that decide the marks

Some important things that the testcases check are:

- dequeuing from an empty buffer, repeatedly, and then using it normally
- filling it exactly, and emptying it exactly (as per the test input)
- one enqueue past full — the first dequeue afterwards gives the *second* element, because the first was overwritten
- enqueueing sixteen values into eight words, so only the last eight survive
- enqueues and dequeues interleaved until the read and write positions have each been round the storage more than once
- negative values and zero. -1 is the empty marker, and a buffer holding a real -1 must still behave
- the vacancy read on an empty buffer, part-filled, full, and after draining — and across an overwrite, where it does not move

Your submission will be graded using more testcases than given here. The additional test cases are of similar kind (same rules, but applied over buffers of different sizes, with different values, etc).

► *Two things to keep in mind. An enqueue into a full buffer moves **head**, leaves **count** unmodified, returns -1 , and vacancy remains 0. Incrementing the index by one does not need a division – it can only be larger than size by 1. So a simple compare and subtract would be enough.*

2 Part 2: Extend the simulator to support Control and Status Registers (CSRs)

The single cycle processor simulator given knows only about general purpose integer registers (**x0** to **x31**). A RISC-V core, typically needs to also support other ISA extensions such as the **Zicsr**.

In this part, you are required to extend the simulator by adding a second register file (for CSRs) beside the **x[REGs]** and handle new instructions that read and write them.

2.1 What are CSRs

Control and status registers hold the machine's state, or information *about itself* (rather than information or state of the user program). Unlike the data of user programs, this machine information is not accessible through regular load or store instructions. The only way to access this machine information is through the instructions given in the **Zicsr** extension.

The CSRs are said to be in a separate “address space”. They are addressed by a **12-bit CSR number**, (and not by a memory address). Therefore, we can have only $2^{12} = 4096$ CSRs. An actual RISC-V processor may implement only a few CSRs (of 4096 possible ones).

Our simulator implements only five CSRs, which our `crt.S` requires while it starts our programs (RISCV assembly codes):

Number	Name	What it is for
0x300	<code>mstatus</code>	Machine status.
0x305	<code>mtvec</code>	Trap vector.
0x341	<code>mepc</code>	The PC a trap interrupted.
0x342	<code>mcause</code>	Why the trap fired.
0xF14	<code>mhartid</code>	Which hardware thread this is. Read-only , and always 0 in our case.

The CSR numbers are not consecutive (and intentionally so). Since we are implementing only five CSRs, we will use `enums` instead of implementing it as an array (with CSR number used to index into this array). See `src/csr.h` for more details.

► *Any CSR number other than those five is an error (not a zero). A program reading one is either using a feature that is not modelled or has jumped somewhere that is not code should error.*

2.2 The three instructions

You have to implement the following three instructions (part of the **Zicsr** extension). All three are **read-then-modify**: they all write the destination register with the contents of CSR *before* the instruction, and modify the CSR with something computed from that old value.

Instruction	New CSR value	
<code>csrrw rd, csr, rs1</code>	<code>x[rs1]</code>	Read/write: replace it outright.
<code>csrrs rd, csr, rs1</code>	<code>old x[rs1]</code>	Read and set: every 1 bit in the source sets that bit.
<code>csrrc rd, csr, rs1</code>	<code>old & ~x[rs1]</code>	Read and clear: every 1 bit in the source clears that bit.

In all three, `x[rd]` receives the old value.

When an instruction does *not* write

This is the rule that is easy to get wrong. It handles the **source** register, not the destination:

- **csrrs** and **csrrc** with **rs1 = x0** **must not write at all**. A mask of zero would change nothing. A write usually triggers side effects, but a read does not. Therefore, writing back the old value is not the same as not writing at all, because the former triggers side effects.
- **csrrw** **always writes**, even with **rd = x0**. Writing is the whole purpose of it (always triggers side effects).

This distinction matters because commonly used pseudo-instruction depend on it. The four pseudo-instructions are:

Written	Assembles to
<code>csrr rd, csr</code>	<code>csrrs rd, csr, x0</code>
<code>csrw csr, rs</code>	<code>csrrw x0, csr, rs</code>
<code>csrs csr, rs</code>	<code>csrrs x0, csr, rs</code>
<code>csrc csr, rs</code>	<code>csrrc x0, csr, rs</code>

The encoding

All three instructions are I-type instructions (opcode **SYSTEM** = 1110011), and differ only in **funct3**. The 12-bit CSR number sits where an I-type immediate normally would:

31 – 20	19 – 15	14 – 12	11 – 7	6 – 0
csr	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

Instruction	funct3	opcode
csrrw	001	1110011
csrrs	010	1110011
csrrc	011	1110011

The decoder is already written for you. `decode()` fills in `d.op`, `d.rd`, `d.rs1` and `d.csr` for you, and `is_csr()` says whether an instruction is one of the family. The encoding is here so that you can read a disassembly and check what you are being handed, not because you have to decode it.

2.3 Implementation Guidelines

A CSR is a register *inside* the core. It has no memory address and so **nothing about a CSR access goes near the memory stage**. It uses the two register-file stages twice: it reads `x[rs1]` and a CSR together, and it writes `x[rd]` and that CSR together. In other words, a **Zicsr** instruction reads (and writes) a CSR register as and when the `x[REG]` is read (and written).

	Reads, in ID	Writes, in WB
an ordinary instruction	<code>x[rs1]</code> , <code>x[rs2]</code>	<code>x[rd]</code>
a CSR instruction	as above and the CSR	as above and the CSR

In an actual hardware, the two register files sit side by side and are read and written in parallel (in the same cycle). A single-cycle processor model has no “same cycle” to speak of, so each half is a function called immediately after the register file it accompanies, and `src/processor_single.cpp` already does so:

```
stage_operands(cpu, d, &v_rs1, &v_rs2, &c); // reg_read(), then
    csr_read()
...
stage_writeback(cpu, d, e.result, m, c, next); // reg_write(), then
    csr_write()
```

The order is important: the new value is computed *from* the old one, so the read has to happen first — and `x[rd]` needs that same old value at writeback.

How information is passed between the read half and the write half

`src/processor.h` declares `CsrAccess`, the record the read half fills in and the write half completes. Every call that passes it from one to the other is already written for you:

```
struct CsrAccess {
    bool did_read;    // this instruction named a CSR, and it was read
    bool did_write;   // ...and the write half happened as well
    u32  num;         // which CSR (its 12-bit number)
    u32  value;       // what it held before (what x[rd] receives)
    u32  next;        // what this instruction computed for it
    u32  written;     // what it holds now. Not always next: see mhartid
};
```

What the trace prints

`trace_retire()` is given to you. It now prints a third kind of a message (see below), and this is how we test to see whether you implemented the write rule correctly.

```
... | csr[300] 00000000 <= 0001e000      read, and then wrote
... | csr[f14] 00000000 (read only)      read, and correctly did not
    write
```

2.4 Your tasks

Search for `TODO(midsem, part 2)` or `TODO(midsem)`. There are five places that you will have to change.

File	Function	
<code>src/csr.cpp</code>	<code>csr_index()</code>	which slot a CSR number names
<code>src/csr.cpp</code>	<code>csr_writable()</code>	which registers may be written
<code>src/processor.cpp</code>	<code>csr_read()</code>	the read half, beside <code>x[rs1]</code>
<code>src/processor.cpp</code>	<code>csr_write()</code>	the write half, beside <code>x[rd]</code>
<code>src/processor.cpp</code>	<code>writeback_value()</code>	what <code>x[rd]</code> receives

Write `csr_index()` first. Everything else depends on it. Read `src/csr.h` for what the file holds, and then the section of `src/processor.h` that declares the two halves (directly below `reg_read()` and `reg_write()`).

- *A write to `mhartid` must not change it. On real hardware, such an instruction raises an illegal-instruction trap, but since this simulator does not model traps, we will discard the write. However, the read half still happens, so `x[rd]` is still written, and the instruction still counts as having written.*
- Similarly, a real hardware forces the low bits of `mepc` to zero because it holds the address of an instruction. The simulator just stores whatever is given to it.*
- You DO NOT have to implement any of these simplifications. They are here so that you are not surprised by it.*

2.5 Testing your work

There are three tiers of test cases, and they fail in a useful order:

1. **build** — it compiles.
2. **unit** — 20 tests calling your pieces one at a time: the lookup, the writability rule, the read half on its own, the write half on top of it, and the two halves run through the stages they belong to. Each failure names the case.
3. **programs** — your simulator runs four programs and its trace and counters are compared against ours, instruction by instruction. The programs are in `programs/csr/` and are worth reading: between them they cover every rule above.

Two new counters have been added to `stats.json` — `csr_reads` and `csr_writes` — and they are compared too. They catch an implementation that gets every *value* right while writing when it should not.

3 Files submitted

1. `details.txt` — name, roll number, machine number.
2. `programs/asms/1-fifo.s` — part 1.
3. `src/csr.cpp` and `src/processor.cpp` — part 2.